

GraphBench Challenge

Projet : Réfutation automatique de conjectures en théorie des graphes

Recherche heuristique de contre-exemples pour des conjectures en théorie des graphes

Réalisé par :

Malado SOW

Raphaël MEIMOUN

Année universitaire : 2025-2026

Dépôt GitHub : <https://github.com/maladoooo/graphbench>

Introduction

Le but du projet est de chercher automatiquement des contre-exemples à des conjectures en théorie des graphes. Les conjectures sont données dans un fichier benchmark. Elles ont toutes une forme proche de :

$$Y(G) \leq f(X(G))$$

Où :

$$Y(G) \geq f(X(G))$$

Où X et Y sont des invariants de graphe, et où f est un polynôme donné dans le fichier Excel.

L'objectif n'est pas de recopier les contre-exemples présents dans le benchmark, mais de construire une méthode capable de les retrouver en générant et en modifiant des graphes. Le programme doit aussi rester assez général pour pouvoir traiter de nouvelles conjectures ajoutées par l'enseignant, tant que les invariants utilisés sont implémentés.

Lecture des conjectures

Chaque ligne du benchmark est transformée en objet Conjecture. Cet objet contient les informations nécessaires à la recherche :

- L'identifiant de la conjecture ;
- La classe de graphes concernée, par exemple connected, tree ou claw_free ;
- Les deux invariants X et Y ;
- Le signe de l'inégalité ;
- Les coefficients du polynôme f.

Le programme n'utilise pas le contre-exemple connu comme point de départ. Il sert uniquement à comparer les résultats une fois la recherche terminée.

À partir de ces informations, on définit une fonction de violation. Si la conjecture est $Y(G) \leq f(X(G))$, alors la violation vaut $Y(G) - f(X(G))$. Si la conjecture est $Y(G) \geq f(X(G))$, alors la violation vaut $f(X(G)) - Y(G)$. Dans les deux cas, une valeur strictement positive signifie que le graphe trouvé est un contre-exemple.

Calcul des invariants

Pour évaluer un graphe, le programme calcule les invariants nécessaires à la conjecture étudiée. Cela évite de recalculer tous les invariants du benchmark à chaque essai.

Les principaux invariants pris en charge sont par exemple :

- Le nombre de sommets et d'arêtes ;
- La densité ;
- Les degrés minimum, maximum et moyen ;
- Le nombre de clique ;

- Le nombre de triangles ;
- Les nombres de domination, domination totale et domination indépendante ;
- Le nombre d'indépendance ;
- Le nombre de matching ;
- Le rayon et le diamètre ;
- La proximité et l'éloignement ;
- Certaines valeurs propres.

Certains invariants sont plus coûteux que d'autres. Par exemple, les invariants de domination ou les valeurs propres demandent plus de calcul que la densité ou le degré maximum. Pour cette raison, la recherche limite la taille des graphes lorsque certains invariants deviennent trop chers.

Génération des graphes

La recherche commence par générer des graphes candidats. Une première génération construite des graphes compatibles avec la classe demandée :

- Graphe connexe ;
- Arbre ;
- Graphe biparti ;
- Graphe planaire ;
- Graphe sans griffe.

Ensuite, des générateurs plus ciblés sont utilisés selon les invariants présents dans la conjecture. Par exemple, si la conjecture concerne le nombre de clique, il est naturel de tester des graphes construits autour de cliques. Si elle concerne le diamètre ou le rayon, des cycles, chemins ou graphes de type "balai" sont de bons candidats. Si elle concerne la connectivité algébrique, des graphes formés de deux parties denses reliées par peu d'arêtes peuvent être intéressants.

Cette étape est importante car elle permet de ne pas partir uniquement de graphes aléatoires. Les graphes initiaux sont déjà orientés vers des formes susceptibles de violer la conjecture.

Recherche locale

La partie principale de l'algorithme est une recherche locale. L'idée est de partir d'un graphe, de le modifier légèrement, puis de garder les modifications qui améliorent le score.

Le schéma général est le suivant :

1. Générer une population de graphes candidats.
2. Calculer le score de chaque graphe.
3. Garder le meilleur graphe courant.
4. Appliquer une mutation.
5. Réparer le graphe si la classe n'est plus respectée.

6. Recalculer le score.
7. Si le graphe est meilleur, le conserver.
8. Si la violation est positive, retourner le contre-exemple.

Les mutations utilisées sont simples mais efficaces :

- Ajouter une arête ;
- Supprimer une arête ;
- Déplacer une arête ;
- Ajouter un sommet ;
- Supprimer un sommet ;
- Subdiviser une arête ;
- Ajouter ou retirer une feuille dans le cas des arbres.

Après une mutation, le graphe peut ne plus appartenir à la bonne classe. Par exemple, un graphe connexe peut devenir non connexe si on supprime une arête. Le programme applique donc une étape de réparation pour rétablir les contraintes de la conjecture.

Fonction de score

Le score principal est la violation de la conjecture. Plus la violation est grande, plus le graphe est intéressant. Si la violation devient positive, on a trouvé un contre-exemple.

Pour guider la recherche, une version heuristique du score ajoute quelques critères structurels. L'idée est de ne pas regarder seulement la violation immédiate, mais aussi certaines propriétés qui peuvent aider à se rapprocher d'un contre-exemple.

Par exemple, le score peut favoriser :

- Un diamètre plus grand ;
- Un degré maximum plus grand ;
- Un nombre de triangles plus élevé ;
- Une densité intermédiaire ;
- Des graphes pas trop grands lorsque cela n'apporte rien.

Cette fonction de score n'est pas utilisée comme preuve. Elle sert seulement à guider la recherche. La validation finale reste toujours violation > 0 .

Adaptabilité de l'approche

Un point important du projet est que l'algorithme ne dépend pas d'une conjecture précise. Il lit automatiquement les informations du benchmark : classe de graphes, invariants X et Y, signe de l'inégalité et coefficients du polynôme. À partir de ces données, il construit la fonction de violation adaptée à la conjecture. La même boucle de recherche peut donc être utilisée sur une nouvelle conjecture ajoutée dans le fichier Excel.

La seule limite est que les invariants utilisés par la nouvelle conjecture doivent être implémentés. Si l'enseignant ajoute une conjecture avec un invariant déjà présent dans le programme, elle peut être testée directement. Si un nouvel invariant apparaît, il faut ajouter son calcul dans le module des invariants.

Résultats

Les résultats sont sauvegardés dans un fichier CSV et dans des fichiers JSON détaillés. Pour chaque conjecture, on conserve :

- L'identifiant de la conjecture ;
- Le graphe trouvé au format graph6 ;
- Les valeurs de X , Y et $f(X)$;
- La violation obtenue ;
- Le temps de recherche ;
- Le coût associé.

Sur le benchmark fourni, la version finale du programme réfute les 100 conjectures.

Le fichier results/results.csv indique :

Mesure	Valeur
Conjectures testées	100
Conjectures réfutées	100
Score total	7.186
Temps moyen de réfutation	0.07186 s
Temps maximum	1.171 s

Ces résultats montrent que la combinaison entre génération ciblée, mutations locales et fonction de score est suffisante pour retrouver rapidement des contre-exemples sur ce benchmark.

Afin d'illustrer concrètement les performances de notre approche, voici une capture d'écran des résultats produits par le programme sur les 100 conjectures du benchmark.

```
=====
RÉSULTATS FINAUX – FunSearch (heuristique LLM)
Conjectures réfutées : 100/100
Score total (officiel): 3.503 (somme des ti, 120 si échec)
Temps moyen (trouvées): 0.035s
Plus longue      : #3384 (0.688s)
Plus rapide     : #6269 (0.0000s)
Résultats CSV    : results/results.csv
=====
(.venv) (base) raphaelmeimoun@MacBook-Pro-de-Raphael graphbench-projec
t 2 %
```

Commandes utilisées

Pour installer les dépendances, il suffit d'exécuter `pip install -r requirements.txt`.

Pour tester une conjecture précise, on lance `python -m src.main --id 981 --time 30 --verbose`.

Pour lancer la recherche sur l'ensemble du benchmark : `python -m src.main --all --time 10 --heuristic -stagnation 1500 --output results/results.csv`.

Pour reprendre une exécution interrompue, on ajoute simplement le flag `--resume` à la commande précédente.

Limites

L'approche reste heuristique. Elle ne garantit pas de trouver un contre-exemple pour n'importe quelle conjecture. Elle dépend de la qualité des graphes générés, des mutations disponibles et du temps de recherche.

De plus, certains invariants sont calculés par approximation ou avec des méthodes gloutonnes pour garder un temps de calcul raisonnable. Cela peut être suffisant pour guider la recherche, mais il faut rester prudent si l'on veut faire une vérification mathématique complète.

Enfin, l'adaptation à de nouvelles conjectures est automatique seulement si les invariants nécessaires sont déjà codés.

Conclusion

Nous avons développé une méthode de recherche locale pour réfuter automatiquement des conjectures de graphes. Chaque conjecture est transformée en fonction de violation, puis le programme cherche un graphe qui rend cette violation positive.

L'intérêt principal de l'approche est qu'elle n'est pas écrite pour une seule conjecture. Elle utilise les informations du benchmark pour construire automatiquement le score adapté, puis explore l'espace des graphes par génération, mutation et réparation.

La version finale obtient 100 contre-exemples sur les 100 conjectures du benchmark, ce qui valide l'efficacité de l'heuristique sur les instances fournies.

